

Lab 1: Experiencing Blocking (Row-Level Lock)

Step 1, 2:

dvd_rental/postgres@PostgreSQL 18		dvd_rental/postgres@PostgreSQL 18	
Query Query History <pre>1 -- Step 1 2 BEGIN; 3 UPDATE film SET rental_rate = 4.99 4 WHERE film_id = 1; 5 -- Holds X-lock on film_id = 1 6 7 SELECT rental_rate FROM film 8 WHERE film_id = 10; 9 COMMIT;</pre>		Query Query History <pre>1 -- Step 2 2 BEGIN; 3 UPDATE film SET rental_rate = 5.99 4 WHERE film_id = 1; 5 -- BLOCKS IMMEDIATELY! 6 -- WAITS for session A 7 8 COMMIT;</pre>	
Data Output Messages Notifications UPDATE 1 Query returned successfully in 56 msec.		Data Output Messages Notifications Total rows: Waiting for the query to complete... 00:00:02.439	
CRLF Ln 1, Col 1		CRLF Ln 1, Col 1	

Step 3:

dvd_rental/postgres@PostgreSQL 18		dvd_rental/postgres@PostgreSQL 18	
Query <pre>1 -- Step 1 2 BEGIN; 3 UPDATE film SET rental_rate = 4.99 4 WHERE film_id = 1; 5 -- Holds X-lock on film_id = 1 6 7 -- Step 3 8 COMMIT;</pre>		Query <pre>1 -- Step 2 2 BEGIN; 3 UPDATE film SET rental_rate = 5.99 4 WHERE film_id = 1; 5 -- BLOCKS IMMEDIATELY! 6 -- WAITS for session A 7 8 COMMIT;</pre>	
Data Output Messages Notifications COMMIT Query returned successfully in 72 msec.		Data Output Messages Notifications UPDATE 1 Query returned successfully in 56 secs 813 msec. ✓ Query returned successfully in 56 secs 813 msec. ✕	
Total rows: Query complete 00:00:00.072 CRLF Ln 8, Col 1		Total rows: Query complete 00:00:56.813 CRLF Ln 1, Col 1	

Lab 2: Replicating Non-repeatable Read

Step 1, 2

The screenshot displays the PgAdmin interface with two tabs open, both titled "dvd_rental/postgres@PostgreSQL 18".

Left Tab:

- Query Editor:** Contains three steps:
 - Step 1
 - BEGIN;
 - UPDATE film SET rental_rate = 2.99
 - WHERE film_id = 100;
 -
 - Step 3
 - UPDATE film SET rental_rate = 3.99
 - WHERE film_id = 200;
 - Blocks (Waiting for B)
- Data Output:** Shows "UPDATE 1" and "Query returned successfully in 65 msec."
- Status Bar:** Total rows: Query complete 00:00:00.065 CRLF Ln 1, Col 1

Right Tab:

- Query Editor:** Contains four steps:
 - Step 2
 - BEGIN;
 - UPDATE film SET rental_rate = 2.99
 - WHERE film_id = 200;
 -
 - Step 4
 - UPDATE film SET rental_rate = 3.99
 - WHERE film_id = 100;
 - BOOM! ERROR: deadlock detected
- Data Output:** Shows "UPDATE 1" and "Query returned successfully in 60 msec."
- Status Bar:** Total rows: Query complete 00:00:00.060 CRLF Ln 1, Col 1

Step 3, 4

dvd_rental/postgres@PostgreSQL 18		dvd_rental/postgres@PostgreSQL 18	
Query Query History <pre> 1 -- Step 1 2 BEGIN; 3 UPDATE film SET rental_rate = 2.99 4 WHERE film_id = 100; 5 6 -- Step 3 7 UPDATE film SET rental_rate = 3.99 8 WHERE film_id = 200; 9 -- Blocks (Waiting for B) </pre>		Query Query History <pre> 1 -- Step 2 2 BEGIN; 3 UPDATE film SET rental_rate = 2.99 4 WHERE film_id = 200; 5 6 -- Step 4 7 UPDATE film SET rental_rate = 3.99 8 WHERE film_id = 100; 9 -- BOOM! ERROR: deadlock detected </pre>	
Data Output Messages Notifications <pre> UPDATE 1 Query returned successfully in 12 secs 626 msec. </pre>		Data Output Messages Notifications <pre> ERROR: deadlock detected Process 26904 waits for ShareLock on transaction 1139; blocked by process 29264. Process 29264 waits for ShareLock on transaction 1140; blocked by process 26904. SQL state: 40P01 Detail: Process 26904 waits for ShareLock on transaction 1139; blocked by process 29264. Process 29264 waits for ShareLock on transaction 1140; blocked by process 26904. </pre>	
Total rows: Query complete 00:00:12.626 CRLF Ln 6, Col 1		Total rows: Query complete 00:00:01.079 CRLF Ln 6, Col 1	

Lab 4: Preventing Race Conditions (FOR UPDATE)

Step 1, 2

dvd_rental/postgres@PostgreSQL 18

No limit

Query

Query History

```

1  -- Step 1
2  BEGIN;
3  SELECT * FROM inventory
4  WHERE inventory_id = 10 FOR UPDATE;
5  -- Locks the row proactively
6
7  -- Step 3
8  INSERT INTO rental
9  (rental_date, inventory_id , customer_id, staff_id)
10 VALUES (NOW() , 10, 1, 1);
11 COMMIT;

```

Data Output

Messages

Notifications

COMMIT

Query returned successfully in 54 msec.

Total rows:

Query complete 00:00:00.054

CRLF

Ln 11, Col 1

dvd_rental/postgres@PostgreSQL 18

No limit

Query

Query History

```

1  -- Step 2
2  BEGIN;
3  SELECT * FROM inventory
4  WHERE inventory_id = 10 FOR UPDATE;
5  -- BLOCKS IMMEDIATELY
6
7  -- Step 4
8  -- Unblocks after A commits.
9  -- App logic detects it 's already rented!
10 ROLLBACK;

```

Data Output

Messages

Notifications

ROLLBACK

Query returned successfully in 75 msec.

Total rows:

Query complete 00:00:00.075

CRLF

Ln 7, Col 1